FPGA 디지털 회로 설계 학습 보고서

작성 기준일: 2026-10-26

분석 대상: D:₩project₩fpga₩ai2

대상 플랫폼: Basys3 / Xilinx Artix-7

개발 도구: Vivado 2020.2 / Verilog HDL / XSim

1. 개요

1.1 목적 및 분석 범위

본 보고서는 FPGA 실습 프로젝트의 설계 소스, 테스트벤치, 핀 제약 및 프로젝트 설정을 분석하여 학습 주제와 구현 내용을 체계적으로 기록한 문서이다. 분석 근거는 Verilog 파일, XDC 파일, XPR 파일 및 PWM 모터 프로젝트의 기술 문서이다.

원본의 18개 프로젝트 폴더 중 17개 폴더에 보존 대상 파일이 존재한다. project_1은 설계 소스가 없는 빈 프로젝트이며, 나머지 폴더에는 동일 주제의 확장 및 변형 실습이 포함된다. 프로젝트 수는 독립적인 완성 설계의 수와 구분한다.

학습 순서는 코드의 개념적 의존 관계를 기준으로 재구성하였다. 개별 실습의 실제 수행일, 작성자 및 보드 동작 여부는 파일 분석만으로 확정하지 않는다. 현재 폴더의 강의 PDF와 PPTX는 참고 자료로 보존하며, 본 보고서의 학습 범위는 프로젝트 코드에서 확인되는 내용으로 한정한다.

1.2 개발 환경

- FPGA 보드: Basys3.
- 대상 디바이스: xc7a35tcpg236-1.
- 설계 언어 및 도구: Verilog HDL, Vivado 2020.2, XSim.
- 시스템 클럭: 주요 설계 기준 100 MHz, 주기 10 ns.
- 입력 장치: 슬라이드 스위치 및 푸시 버튼.
- 출력 장치: LED, 4자리 7세그먼트 표시기(FND), PMOD PWM 및 방향 제어 신호.

1.3 학습 단계

1. 논리 게이트 및 조합논리 표현.
2. 반가산기·전가산기 및 다비트 가산기 구성.
3. 조합 ROM과 주소 디코딩.
4. 플립플롭, 버튼 디바운싱 및 입력 이벤트 처리.
5. Tick 생성과 시간 단위 카운팅.
6. Moore·Mealy FSM 및 시퀀스 검출.

7. 모드 제어, 카운터 및 FND 표시.

8. 시계·스톱워치 시스템 통합.

9. PWM 듀티 조절, 모터 방향 제어 및 출력 검증.

2. 핵심 학습 내용

2.1 조합논리 및 계층적 설계

연속 할당문 assign을 이용하여 논리 게이트와 산술 연산 회로를 표현하였다. 포트 선언, 비트 벡터, 내부 배선 및 이름 기반 포트 매핑을 사용하였으며, 하위 모듈을 인스턴스화하여 상위 기능을 구성하였다. 반가산기에서 전가산기, 4비트 및 7비트 가산기로 확장하는 과정에서 모듈 재사용과 carry 연결 구조를 확인하였다.

2.2 순차논리 및 HDL 할당 방식

클럭 상승 에지와 비동기 리셋을 기준으로 플립플롭, 카운터 및 상태 레지스터를 구현하였다. 순차논리의 nonblocking 할당(<=)과 조합논리의 blocking 할당(=)을 구분하고, always @(*)와 기본값 할당을 통한 조합논리의 완전성을 검토하였다. 일부 초기 구현에는 불완전한 감지 목록과 순차 블록의 blocking 할당이 남아 있다.

2.3 비동기 입력 처리

버튼 입력 처리 구조를 동기화, 디바운싱, 에지 검출로 구분하였다. 동기화는 비동기 입력의 메타안정성 전파 가능성을 낮추며, 디바운싱은 채터링으로 발생하는 반복 변화를 제거한다. 에지 검출은 유지되는 입력 레벨을 단일 이벤트로 변환한다. 실습에는 저속 샘플링, 시프트 레지스터, 안정 시간 카운터 및 이전 입력값 비교 방식이 포함된다.

2.4 시간 기준 생성

카운터를 이용하여 주기적인 Tick을 생성하고, Tick 누적 횟수로 표시 갱신과 시간 계수를 제어하였다. 100 MHz 기준 1 ms는 100,000클럭, 10 ms는 1,000,000클럭에 해당한다. 초기 설계의 분주 신호 기반 클럭 구조와 후속 설계의 시스템 클럭 및 Tick enable 구조를 비교할 수 있다.

2.5 FSM 및 기능 분리

상태 레지스터, 다음 상태 결정 및 출력 논리를 분리하여 FSM을 구성하였다. Moore 방식은 상태에 의해 출력을 결정하며, Mealy 방식은 상태와 입력에 의해 출력을 결정한다. 중첩 가능한 101 시퀀스 검출을 구현하고, 상태 기반 제어를 카운터 모드 및 스톱워치 실행·정지 제어에 적용하였다.

3. 조합논리 및 산술 회로 구현

3.1 기본 논리 게이트 — 01.gate_test

학습 목표: 기본 논리 연산의 HDL 표현과 입력 조합별 출력 확인.

구현 내용: gates.v에서 입력 a, b에 대한 AND, OR, NAND, NOR, XOR 및 NOT 결과를 6비트 LED 출력에 할당하였다. tb_gates.v에는 00, 01, 10, 11 입력 조합을 순차 인가하는 자극이 작성되어 있다.

검토 결과: 테스트벤치의 2비트 i_a가 1비트 a 포트에 연결되어 있으며, b 포트는 미연결 상태이다. 두 입력을 각각 연결하는 수정이 선행되어야 진리표 검증이 유효하다.

3.2 반가산기 및 전가산기 — 02.adder

학습 목표: 합과 자리올림의 논리식 도출 및 하위 모듈 재사용.

구현 내용: half_adder.v에서 sum = a ^ b, cout = a & b를 구현하였다. full_adder.v는 반가산기 두 개와 OR 연산으로 입력 a, b, cin의 합과 carry를 생성한다. tb_half_adder.v는 반가산기의 전체 4개 입력 조합을 인가한다.

검증 범위: 반가산기 입력 자극이 존재한다. 전가산기의 전체 8개 입력 조합에 대한 자동 기대값 비교는 해당 테스트벤치에 포함되지 않는다.

3.3 4비트 리플 캐리 가산기 — 03.adder4

학습 목표: 비트별 carry 전달을 이용한 다비트 산술 회로 구성.

구현 내용: adder4.v에서 전가산기 네 개를 직렬 연결하였다. 각 비트의 carry 출력은 상위 비트의 cin으로 전달된다. 연산 결과는 {cout, sum}으로 구성하여 최상위 carry를 포함한다.

검증 범위: tb_adder4.v에 일반 덧셈, carry 발생 및 최대 입력 조합 등의 자극이 포함되어 있다. test_bed.v는 기능이 작성되지 않은 템플릿이다.

3.4 7비트 가산기 — adder7 / 999.7bit_full_adder

학습 목표: 산술 회로의 비트 폭 확장 및 내부 신호 연결 관리.

구현 내용: adder7.v에서 전가산기 일곱 개를 연결하여 7비트 입력 두 개와 carry 입력을 처리하였다. 두 폴더는 유사 실습의 변형으로 보존하였다.

검토 결과: adder7에는 하위 반가산기 소스가 포함되어 있다. 999.7bit_full_adder에는 필요한 half_adder 소스가 없으며, XPR이 참조하는 imports/new/full_adder.v도 원본에서 누락되어 있다. 해당 변형은 소스 의존성 복구가 필요하다.

3.5 조합 ROM — ROM_always

학습 목표: 주소 디코딩과 상수 테이블 기반 출력 구현.

구현 내용: rom_always.v에서 4비트 주소를 8비트 데이터로 변환하였다. 주소 0, 1, 2, 3, E, F에 상수를 지정하고 나머지 주소에는 기본값 0을 출력한다. always @(*)와 case를 이용한 조합 ROM 구조이다.

검증 자료: rom_always_tb.v가 포함되어 있다. 본 정리 작업에서 해당 테스트는 재실행하지 않았다.

4. 버튼 입력 및 시간 제어 구현

4.1 D 플립플롭 기반 입력 처리 — 06.dff_btn_debounce

학습 목표: 분주, 플립플롭 샘플링, 에지 검출 및 출력 토글의 연결.

구현 내용: clock_80Hz.v에서 100 MHz를 약 80 Hz로 분주하였다. 두 D_FF의 출력에 Q1 & ~Q2를 적용하여 눌림 펄스를 생성하고, led_toggle.v에서 LED 상태를 반전한다. 80 Hz의 주기는 12.5 ms이다.

검토 결과: 일부 주석의 80 MHz 표기는 실제 분주값과 일치하지 않는다. 분주 출력과 버튼 펄스를 별도 클럭으로 사용하는 구조이므로, 시스템 클럭 및 enable 기반 구조로의 개선을 검토할 수 있다.

4.2 안정 시간 기반 디바운서 — 06_1.btn_debounce_10ms

학습 목표: 일정 시간 유지된 입력만 유효한 상태로 반영하는 디바운싱 구현.

구현 내용: DEBOUNCE_LIMIT = 999_999를 적용하여 100 MHz 기준 약 10 ms의 안정 시간을 설정하였다. 최상위 모듈은 디바운서 출력을 LED 토글 회로에 연결한다. new 경로의 구현에는 2단 플립플롭 동기화가 포함되어 있다.

검토 결과: new와 imports/Downloads에 동일한 debouncer 이름의 변형 소스가 존재하므로 XPR에 등록된 파일 기준의 소스 선택이 필요하다. new 버전은 최종 상태 갱신에 원시 입력 noisy_btn를 사용하므로 동기화된 r_FF2 사용 여부를 검토해야 한다.

4.3 시프트 레지스터 기반 디바운서 — my_btn_debounce

학습 목표: 입력 선택과 연속 샘플 이력을 이용한 버튼 상태 판정.

구현 내용: btn_mux.v에서 입력 버튼을 선택하고, 1 ms 샘플링과 8비트 시프트 레지스터를 이용하여 연속된 동일 입력을 판정한다. top_btn.v는 버튼 상태 및 토글 출력을 연결하며, tb_top_btn.v는 입력 자극을 제공한다.

검토 결과: nonblocking 갱신 이전의 r_shift를 판정하므로 출력 반영 시 추가 샘플 지연이 발생할 수 있다. 외부 XDC는 복사본의 external/로 보존하였으나, 활성 제약의 a, b, ld0 포트가 현재 최상위 모듈의 포트와 일치하지 않는다.

4.4 Tick 기반 LED 제어 — my_tick_ledonoff

학습 목표: 주기 펄스 누적에 의한 시간 제어.

구현 내용: tick_generator.v와 led_onoff_500ms.v가 포함되어 있다. LED 제어기는 1 ms Tick을 500회 계수한 후 출력을 반전한다. 점등 및 소등 시간은 각각 500 ms이며 전체 주기는 1초이다.

검토 결과: XPR의 설계 Top은 top_btn_debounce로 지정되어 있다. 신규 LED 제어 모듈의 최상위 연결은 추가 통합이 필요하다.

5. 상태기계 및 표시 시스템 구현

5.1 Moore 시퀀스 검출기 — 07.moore_fsm_101_detect

학습 목표: 상태 기반 출력과 중첩 시퀀스 검출.

구현 내용: moore_fsm_101.v에서 대기, 1 검출, 10 검출, 101 검출의 네 상태를 정의하였다. 검출 상태 S3에서 출력을 활성화하며, S3에서 0 입력 시 S2로 전이하여 중첩을 허용한다. tb_moore_fsm_101.v에는 기대값 비교와 PASS/FAIL 출력이 포함되어 있다.

검토 결과: 최상위 모듈의 디바운서 출력은 w_clean_btn이지만 FSM 입력은 clean_btn으로 연결되어 있다. 코어 단위 테스트와 별도로 최상위 신호 연결 수정이 필요하다.

5.2 Mealy 검출기 및 출력 시점 비교 — project_2 / mealy_fsm

학습 목표: Moore와 Mealy의 상태 구성 및 출력 결정 시점 비교.

구현 내용: project_2/mealy_fsm.v는 세 상태로 101을 검출하고 상태와 입력에 따라 조합 출력을 생성한다. 테스트 파일명에는 Moore가 포함되지만 내부 모듈명은 tb_mealy_fsm_101이다.

mealy_fsm 프로젝트는 button_step.v, moore_fsm.v, mealy_fsm.v 및 top_fsm_101_basys3.v로 구성된다. 버튼 이벤트 단위의 비트 입력을 두 FSM에 적용하며, tb_fsm_101_compare.v는 클럭 전후의 출력 관찰 시점과 중첩 검출 결과를 비교한다.

검증 결과: mealy_fsm의 비교 테스트는 XSim 재실행에서 PASS를 확인하였다. project_2 테스트는 Mealy 출력의 관찰 시점과 기대값 일치 여부에 대한 추가 검토가 필요하다.

5.3 증가·감소 카운터 및 FND — 09.up_down_counter

학습 목표: 모드 전환, 주기 계수, 이진수의 십진 자릿값 분리 및 다중화 표시.

구현 내용: main_controller.v에서 IDLE, 증가, 감소 및 스위치 읽기 모드를 제어한다. 10 ms 단위 계수와 0000~9999 순환 및 표시 데이터 선택을 구현하였다. fnd_controller.v는 나눗셈·나머지 연산으로 자릿값을 분리하고 active-low 자리 선택 및 숫자별 세그먼트 패턴을 출력한다.

검토 결과: 순차 블록의 blocking 할당, posedge tick 기반 자리 선택 및 always @(sel)의 불완전한 감지 목록이 확인되었다. 자리 선택이 유지된 상태에서 데이터만 변경되는 조건의 표시 갱신을 점검해야 한다.

5.4 시계·스톱워치 통합 — stopwatch

학습 목표: 입력 처리, 상태 제어, 시간 계수 및 표시 기능의 모듈별 분리와 통합.

시스템 구성: 버튼 디바운서 → button_edge → mode_controller → time_counter → BCD 변환 → FND 출력. 시간 계수에는 10 ms Tick, 표시 스캔에는 1 ms Tick을 사용한다.

주요 모듈:

• mode_controller.v: CLOCK, STOPWATCH, PAUSE 상태 제어. Left는 CLOCK과 실행 상태 사이의 전환, Right는 실행·정지 전환, Center는 실행 중 초기화에 사용된다. PAUSE에서 Left로 CLOCK에 복귀하는 전이는 구현되지 않았다.

• time_counter.v: 시계의 분·초와 스톱워치의 초·센티초를 계수한다. 시계 범위는 00:00~59:59이며, 스톱워치 초는 59 이후 0으로 순환한다.

• button_edge.v: clean_btn & ~prev_btn를 이용하여 한 클럭 길이의 버튼 이벤트를 생성한다.

• ani_controller.v: PAUSE 상태에서 5초 동안 입력이 없으면 100 ms 간격으로 12단계 애니메이션을 수행한다.

• top.v: BCD 분리, 표시 모드 선택 및 소수점 제어를 통합한다. 시계 소수점은 1초마다 반전하며 전체 점멸 주기는 2초이다.

검증 자료: tb_button_edge.v, tb_tick_generator.v, tb_stopwatch.v. tb_stopwatch.v는 모드 제어기 단위 테스트이며 전체 시스템 테스트가 아니다. XPR의 기본 Simulation Top은 tb_tick_generator이다.

6. PWM 및 모터 제어 구현

6.1 PWM 듀티 제어 — 10.pwm_control

학습 목표: 카운터와 비교기를 이용한 PWM 생성 및 버튼 기반 듀티 조절.

구현 내용: pwm_duty_control.v에서 0~9 카운터와 듀티 설정값을 비교하여 PWM 출력을 생성한다. 디바운서, Tick 생성기, FND 표시기 및 스위치 기반 방향 출력을 최상위 모듈에 연결하였다.

검토 결과: 8비트 스위치 신호와 2비트 방향 관련 포트 사이의 폭 차이가 존재한다. 표시값의 단위와 방향 표시 구현에 대한 점검이 필요하다.

6.2 모터 방향 및 백분율 표시 — pwm_motor

학습 목표: PWM 제어의 시스템 확장, 방향 표시, 측정 신호 제공 및 자동 검증.

구현 내용: 기본 PWM 구조를 확장하여 듀티 백분율, 방향 문자, 방향 문자 점멸 및 오실로스코프용 PWM 복제 출력을 구현하였다. tests/tb_motor.v, check_synthesis.tcl, capture_waveform.tcl이 포함되어 있다.

제어 사양:

- 초기 듀티: 50%.
- 조절 범위: 0~90%, 단계 간격 10%. 100% 단계는 미구현.
- 입력 처리: 증가·감소 버튼의 상승 에지를 검출하여 유지 입력의 반복 변경을 방지.
- 방향 선택: 논리 스위치 10은 정회전 및 F 표시, 01은 역회전 및 b 표시. 00과 11은 방향 문자 공백.
- 정지 설정: 방향선 11에서도 PWM 출력과 듀티 설정은 유지.
- 표시 형식: 듀티 단계에 10을 곱하여 050, 090 등의 백분율로 표시.
- 방향 문자 점멸: 1초 점등 및 1초 소등, 전체 주기 2초.

출력 및 핀 구성: PWM은 J1/JA1, IN1은 L2/JA2, IN2는 J2/JA3, 측정용 PWM 복제는 A15/JB7에 할당된다. 논리 sw[1]은 V16에 연결되어 물리 SW3에 대응한다.

파형 계산: 시스템 클럭 100 MHz를 10회 계수하므로 PWM 주파수는 10 MHz, 주기는 100 ns이다. 듀티 10%, 50%, 90%의 HIGH 시간은 각각 10 ns, 50 ns, 90 ns이다. 듀티 0%에서는 출력이 LOW로 유지된다.

검증 결과: XSim에서 듀티 범위, 상·하한, 버튼 유지 입력, 방향 및 표시 갱신, 점멸, 리셋 항목의 PASS를 확인하였다. 테스트 대상은 PWM 및 FND 관련 모듈이며, 최상위 전체 입력 경로와 실제 모터 동작은 검증 범위에 포함되지 않는다. 기존 문서의 실측 항목은 미측정 상태이다.

7. 검증 결과 및 개선 과제

7.1 검증 수준

테스트벤치에는 initial, 시간 지연, 클럭 생성, task, 기대값 비교 및 종료 처리가 사용되었다. 가산기 등 일부 테스트는 입력 자극과 파형 관찰 중심이며, FSM 및 모터 테스트에는 자동 기대값 비교가 포함된다.

본 정리 과정에서 재실행한 테스트는 pwm_motor/tests/tb_motor.v와 mealy_fsm의 tb_fsm_101_compare.v이며 두 항목 모두 PASS를 확인하였다. 전체 프로젝트의 합성, 구현, 타이밍 분석, 비트스트림 생성 및 보드 실측은 수행하지 않았다. 세부 결과와 재실행 방법은 VALIDATION.md에 기록하였다.

7.2 개선 우선순위

1. 최상위 포트 연결 오류, 미연결 입력 및 누락 하위 모듈 해결.
2. 프로젝트별 설계 Top, Simulation Top 및 XDC 포트 정합성 확인.
3. 테스트벤치의 기대값과 Moore·Mealy 출력 관찰 시점 정합성 확인.
4. 시스템 클럭과 Tick enable 중심의 동기식 구조 정비.
5. 조합 블록 감지 목록, 기본값 및 순차 할당 방식 정비.
6. 경계값 자동 비교, 전체 시스템 검증 및 보드 실측 결과 보완.

8. 산출물 및 저장소 구성

projects/에 소스, 테스트벤치, 핀 제약, XPR, 파형 설정 및 사용자 작성 테스트 스크립트 등 총 123개 파일을 보존하였다. 전체 목록은 CODE_INDEX.md, 파일별 원본 경로와 SHA-256은 COPY_MANIFEST.csv에 기록하였다.

학습 코드의 기능은 변경하지 않았다. my_btn_debounce의 외부 XDC를 내부 external/ 경로에 복사하고 복사본 XPR 참조를 조정하였다. 동일 모듈명의 프로젝트별 변형은 별도로 유지하였다.

Git 관리 대상은 README.md, .gitignore, docs/, projects/, scripts/이다. Vivado 캐시, 합성·구현 결과, 실행 로그, 시뮬레이션 파형 및 비트스트림은 제외한다. 기존 루트의 강의 PDF·PPTX는 로컬 보관 대상으로 설정하며, 본 보고서의 PDF는 문서 산출물로 관리한다.

9. 주요 코드 및 동작 해설

본 절은 보존된 소스의 핵심 구문을 발췌하여 구현 원리와 신호 흐름을 설명한다. 원본의 식별자와 HDL 구문을 유지하되 주석과 주변 선언 일부는 생략하였다. 부분 발췌는 독립 컴파일 단위가 아니며 전체 코드와 소스 의존성은 출처 파일 및 CODE_INDEX.md를 기준으로 확인한다.

9.1 논리 게이트의 연속 할당

출처: gates.v, 원본 11~16행.

```
assign led[0] = a & b;
assign led[1] = a | b;
assign led[2] = ~(a & b);
assign led[3] = ~(a | b);
assign led[4] = a ^ b;
assign led[5] = ~a;
```

동작 해설: 입력 a, b의 변화가 각 논리식에 반영되어 LED 출력으로 전달된다. &와 |는 AND와 OR, ^는 XOR, ~는 비트 반전을 나타낸다. 예를 들어 a=1, b=0이면 AND 출력은 0, OR와 XOR 출력은 1이다. 클럭이나 상태 저장 없이 입력으로 출력이 결정되는 조합회로이다.

9.2 전가산기의 계층적 구성

출처: full_adder.v, 원본 11~26행.

```
half_adder u1_half_adder(
        .a(a),
        .b(b),
        .sum(w_sum1),
        .cout(w_cout1)
);

half_adder u2_half_adder(
        .a(w_sum1),
        .b(cin),
        .sum(w_sum2),
        .cout(w_cout2)
);

assign sum = w_sum2;
assign cout = w_cout1 | w_cout2;
```

동작 해설: 첫 번째 반가산기는 a+b의 부분 합과 carry를 생성한다. 두 번째 반가산기는 부분 합에 cin을 더한다. 최종 합은 w_sum2이며 두 단계 중 어느 단계에서든 carry가 발생하면 cout을 활성화한다. 입력 a=1, b=1, cin=1이면 {cout,sum}=2'b11로 십진수 3을 나타낸다. 다비트 가산기는 이 carry를 다음 비트의 cin에 연결하는 방식으로 확장된

다.

9.3 2단 플립플롭 입력 동기화

출처: button_debounce_10ms.v, 원본 17~26행.

```
reg r_FF1, r_FF2;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_FF1 <= 0;
        r_FF2 <= 0;
    end else begin
        r_FF1 <= noisy_btn;
        r_FF2 <= r_FF1;
    end
end
```

동작 해설: r_FF1은 비동기 버튼 입력을 샘플링하고 r_FF2는 이전 클럭의 r_FF1을 저장한다. nonblocking 할당에 의해 두 레지스터는 이전 값을 기준으로 갱신된다. 이 구조는 메타안정성 전파 가능성을 낮추며 채터링 제거는 별도 안정 시간 판정 회로가 담당한다.

9.4 버튼 상승 에지 검출

출처: button_edge.v, 원본 12~22행.

```
reg [2:0] prev_btn;

always @(posedge clk or posedge reset) begin
    if (reset)
        prev_btn <= 3'b000;
    else
        prev_btn <= clean_btn;
end


assign btn_pulse = clean_btn & ~prev_btn;
```

동작 해설: prev_btn은 이전 클럭의 버튼 상태를 저장한다. 현재 입력이 1이고 이전 값이 0인 비트에서만 btn_pulse가 1이 된다. 버튼을 계속 누르면 다음 클럭부터 두 값이 모두 1이므로 추가 펄스가 발생하지 않는다. 입력이 시스템 클럭에 동기화되어 있다는 조건에서 한 클럭 길이의 이벤트를 생성한다.

9.5 파라미터 기반 Tick 생성

출처: tick_generator.v, 원본 4~23행.

```
parameter INPUT_FREQ = 100_000_000;
    parameter TICK_Hz = 100;
    parameter TICK_COUNT = INPUT_FREQ / TICK_Hz;

    reg[$clog2(TICK_COUNT)-1:0] r_tick_counter = 0;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            tick <= 0; r_tick_counter <= 0;
        end else begin
            if(r_tick_counter == TICK_COUNT - 1) begin
                r_tick_counter <= 0;
                tick <= 1'b1;
            end else begin
                r_tick_counter <= r_tick_counter + 1;
                tick <= 1'b0;
            end
```

```
        end
    end
endmodule
```

동작 해설: TICK_COUNT는 출력 Tick 사이에 필요한 시스템 클럭 수이다. 기본값은 100,000,000/100=1,000,000으로 10 ms 간격의 Tick을 생성한다. 카운터가 TICK_COUNT-1에 도달하면 0으로 복귀하고 tick을 한 클럭 동안 활성화한다. $clog2는 계수에 필요한 비트 폭 산정에 사용된다.

설계 조건: 양의 정수 분주를 전제로 한다. 현재 비트 폭 선언은 TICK_COUNT=1에서 별도 처리가 필요하며, 정확한 주기는 입력 주파수가 Tick 주파수로 정수 분주될 때 성립한다. 아래 발췌에는 원본 모듈의 종료 구문이 포함되어 있으며 포트 선언은 생략되어 있다.

9.6 Moore FSM의 중첩 시퀀스 검출

출처: moore_fsm_101.v, 원본 27~38행.

```
always @(*) begin
    case (curr_state)
        S0: next_state = x ? S1 : S0;
        S1: next_state = x ? S1 : S2;
        S2: next_state = x ? S3 : S0;
        S3: next_state = x ? S1 : S2;
        default: next_state = S0;
    endcase
end


assign y = (curr_state == S3);
```

동작 해설: S0은 대기, S1은 1 검출, S2는 10 검출, S3는 101 검출 상태이다. S2에서 1을 받으면 다음 상승 에지에 S3로 전이하고 y가 활성화된다. S3에서 0을 받으면 S2로 전이하므로 10101의 두 번째 101도 검출한다. 출력은 현재 상태만으로 결정된다.

설계 구분: 발췌는 다음 상태 및 출력 논리이다. 현재 상태를 저장하는 순차 블록은 원본의 별도 블록에 정의되어 있다. 최상위 모듈의 입력 신호 불일치는 이 코어와 별도로 수정해야 한다.

9.7 FND용 십진 자릿값 분리

출처: fnd_controller.v, 원본 31~34행.

```
assign d1 = in_data % 10;
assign d10 = (in_data / 10) % 10;
assign d100 = (in_data / 100) % 10;
assign d1000 = (in_data / 1000) % 10;
```

동작 해설: 10으로 나눈 나머지로 일의 자리를 구하고, 10·100·1000으로 나눈 결과에 다시 나머지 연산을 적용하여 각 자릿값을 추출한다. 입력 1234의 결과는 d1000=1, d100=2, d10=3, d1=4이다. 이후 자리 선택 회로가 각 자릿값을 순차적으로 세그먼트에 출력한다. 이 발췌는 숫자 분리 회로이며 실제 자리 스캔과 세그먼트 디코딩은 별도 로직이다.

9.8 스톱워치 실행 및 초기화 조건

출처: mode_controller.v, 원본 63~67행.

```
assign mode = state;

assign sw_run = (state == S_STOPWATCH);

assign sw_clear = (state == S_STOPWATCH) && btn_center;
```

동작 해설: mode는 현재 상태를 외부에 제공한다. sw_run은 STOPWATCH 상태에서만 1이며 시간 계수기의 실행 조건으로 사용된다. sw_clear는 STOPWATCH 상태와 Center 버튼 이벤트가 동시에 유효할 때 활성화된다. 따라서 현재 구현은 PAUSE 상태에서 Center를 입력해도 초기화하지 않는다.

9.9 센티초 계수와 자리올림

출처: time_counter.v, 원본 49~74행.

```
always @(posedge clk or posedge reset) begin
        if (reset) begin
            stopwatch_sec <= 0;
            stopwatch_cs <= 0;
        end
        else if (sw_clear) begin
            stopwatch_sec <= 0;
            stopwatch_cs <= 0;
        end
        else if (sw_run && tick_10ms) begin
            if (stopwatch_cs == 99) begin
                stopwatch_cs <= 0;

                if (stopwatch_sec == 59)
                    stopwatch_sec <= 0;
                else
                    stopwatch_sec <= stopwatch_sec + 1;
            end
            else begin
                stopwatch_cs <= stopwatch_cs + 1;
            end
        end
    end

endmodule
```

동작 해설: 리셋, 초기화, 실행 및 Tick의 순서로 우선순위가 적용된다. 실행 상태에서 10 ms Tick을 받을 때마다 센티초를 증가시키며, 99 이후에는 0으로 복귀하고 초를 증가시킨다. 초는 59 이후 0으로 순환한다. sw_run=0이면 계수 값을 유지하여 일시정지를 구현한다.

시간 특성: 실행 제어는 공용 10 ms Tick을 기준으로 한다. 실행 재개 후 첫 계수까지의 지연은 Tick 위상에 따라 달라질 수 있다. 발췌에는 원본 모듈 종료 구문이 포함되어 있으며 시계 계수 블록과 포트 선언은 생략하였다.

9.10 PWM 듀티 조절과 유지 입력 억제

출처: pwm_duty_control.v, 원본 20~39행.

```
wire w_duty_inc = (clean_btn[0] && !r_prev_duty_inc);
wire w_duty_dec = (clean_btn[1] && !r_prev_duty_dec);


always @(posedge clk, posedge reset) begin
    if (reset) begin
        r_duty_cycle <= 4'd5;
        r_prev_duty_inc <= 0;
        r_prev_duty_dec <= 0;
    end else begin
        r_prev_duty_inc <= clean_btn[0];
        r_prev_duty_dec <= clean_btn[1];
        if (w_duty_inc && r_duty_cycle < 4'd9)
            r_duty_cycle <= r_duty_cycle + 1;
        if (w_duty_dec && r_duty_cycle >= 4'd1)
            r_duty_cycle <= r_duty_cycle - 1;
    end
end
```

동작 해설: 현재 버튼과 이전 버튼을 비교하여 증가·감소 이벤트를 검출한다. 리셋 시 듀티 단계는 5로 설정되며, 증가 조건은 9 미만, 감소 조건은 1 이상으로 제한한다. 버튼이 유지되는 동안 이전 값도 1이 되므로 반복 조절이 억제된다.

검토 사항: 증가와 감소 조건이 독립된 두 if로 작성되어 있다. 두 버튼이 동시에 유효하고 감소 조건도 만족하면 동일 레지스터에 대한 후행 감소 할당이 우선한다. 동시 입력 정책을 변경할 경우 명시적인 우선순위 또는 무시 조건이 필요하다.

9.11 카운터 비교 방식의 PWM 출력

출처: pwm_duty_control.v, 원본 39~51행.

```
always @(posedge clk, posedge reset) begin
    if (reset) begin
        r_pwm_counter <= 0;
    end else begin
    if (r_pwm_counter >= 4'd9)
        r_pwm_counter <= 0;
        else r_pwm_counter <= r_pwm_counter + 1;
    end
end
assign pwm_out = (r_pwm_counter < r_duty_cycle) ? 1'b1 : 1'b0;
assign pwm_out_led = pwm_out;
assign duty_cycle = r_duty_cycle;
```

동작 해설: r_pwm_counter는 0~9를 반복한다. 카운터가 듀티 단계보다 작을 때만 출력이 HIGH이므로 단계 5는 10클럭 중 5클럭 HIGH, 즉 50%가 된다. 시스템 클럭 100 MHz에서 한 주기는 100 ns이며 주파수는 10 MHz이다. 단계 0에서는 비교 조건이 성립하지 않아 LOW로 유지된다.

갱신 특성: 듀티값은 PWM 주기 경계에 고정하여 갱신하지 않으므로 변경 시점에 해당하는 한 주기의 펄스 폭은 과도 상태를 포함할 수 있다. 정상상태 듀티와 설정 변경 직후의 파형을 구분하여 관찰해야 한다.

9.12 Moore·Mealy 출력의 자동 비교

출처: tb_fsm_101_compare.v, 원본 35~59행.

```
task send_data;
    input a, expect_moore, expect_mealy;
    begin

        x = a;
        en = 1'b1;
        #1;

        $display("t=%0t x=%b Moore=%b Mealy=%b",
                 $time, x, moore_y, mealy_y);

        if (moore_y !== expect_moore ||
            mealy_y !== expect_mealy) begin
            r_error = 1'b1;
            $display("FAIL: expected Moore=%b Mealy=%b",
                     expect_moore, expect_mealy);
        end


        @(posedge clk);
        #1;

        en = 1'b0;
    end
endtask
```

동작 해설: 입력 x와 유효 신호 en을 설정한 후 지연을 두고 두 FSM의 출력을 기대값과 비교한다. !==는 X 또는 Z를 포함한 불일치도 검출한다. 비교 이후 다음 상승 에지를 통과시켜 상태를 전이하고 en을 해제한다. Mealy의 입력 반영과 Moore의 상태 반영을 서로 다른 시점에서 관찰하는 검증 구조이다.

검증 결과: 해당 테스트는 XSim 재실행에서 PASS를 확인하였다. 관찰 시점이 정의된 테스트이며, 실제 비동기 입력에서 발생할 수 있는 출력 글리치나 보드 타이밍까지 검증하는 항목은 아니다.